

COURSE CODE/TITLE:	CPE 212	SECTION:	CPE31S2
DATE PERFORMED:	Sep 1, 2026	DATE SUBMITTED:	Sep 3, 2026
NAME:	Loterte, Mark David M.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 6 Targeting Specific Nodes

1. PROCEDURE OUTPUT

Task 1: Targeting Specific Nodes

1. Create a new playbook and named it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit.



```

mdm1ot@workstation: ~/CPE212_loterte
GNU nano 7.2 site.yml *
--
hosts: all
become: true
tasks:

- name: install apache and php for ubuntu servers
  apt:
    name:
      - apache2
      - libapache2-mod-php
    state: latest
    update_cache: yes
  when: ansible_distribution == "Ubuntu"
- name: install apache and php for CentOS servers
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"
  
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

2. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:

```
mdmlot@workstation: ~
GNU nano 7.2 /etc/ansible/hosts
[web_servers]
192.168.56.104
192.168.56.103
[db_servers]
192.168.56.106
[file_servers]
192.168.56.107

[ Wrote 7 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

Make sure to save the file and exit.

Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.

3. Edit the site.yml by following the image below:

```
mdmlot@workstation: ~/CPE212_loterte
GNU nano 7.2 site.yml *
become: true
pre_tasks:
- name: install updates (CentOS)
  dnf:
    update_only: yes
    update_cache: yes
  when: ansible_distribution == "CentOS"

- name: install updates (Ubuntu)
  apt:
    upgrade: dist
    update_cache: yes
  when: ansible_distribution == "Ubuntu"

- hosts: web_servers
  become: true
  tasks:

- name: install apache and php for ubuntu servers
  apt:
    name:
      - apache2
      - libapache2-mod-php
    state: latest
    update_cache: yes
  when: ansible_distribution == "Ubuntu"

^G Help      ^O Write Out ^W Where Is  ^K Cut      ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

Make sure to save the file and exit.

The pre-tasks command tells the ansible to run it before any other thing. In the pre-tasks, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at web_servers. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.

```
mdm1ot@workstation:~/CPE212_Loterte$ ansible-playbook site.yaml --ask-become-pass
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.107]
ok: [192.168.56.103]
ok: [192.168.56.104]
ok: [192.168.56.106]

TASK [install updates (CentOS)] *****
skipping: [192.168.56.104]
skipping: [192.168.56.106]
skipping: [192.168.56.107]
ok: [192.168.56.103]

TASK [install updates (Ubuntu)] *****
skipping: [192.168.56.103]
changed: [192.168.56.106]
changed: [192.168.56.104]
changed: [192.168.56.107]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [install apache and php for ubuntu servers] *****
skipping: [192.168.56.103]
ok: [192.168.56.104]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.104]
changed: [192.168.56.103]

PLAY RECAP *****
192.168.56.103      : ok=4    changed=1    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
192.168.56.104      : ok=4    changed=1    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
192.168.56.106      : ok=2    changed=1    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0
192.168.56.107      : ok=2    changed=1    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0
```

Run the site.yml file and describe the result.

The site.yml file ran perfectly. However, due to my current Ubuntu releases lacking a lot of updates, it took a considerably longer time than usual. Because hosts was only set to web_servers, the task was only done to 192.168.56.103 and 192.168.56.104, following my designation in the hosts inventory file.

4. Let's try to edit again the site.yml file. This time, we are going to add plays targeting the other servers. This time we target the db_servers by adding it on the current site.yml. Below is an example: (Note add this at the end of the

playbooks from task 1.3.

```

GNU nano 7.2                                site.yaml
- httpd
- php
  state: latest
when: ansible_distribution == "CentOS"

- hosts: db_servers
  become: true
  tasks:
    - name: install mariadb package (CentOS)
      yum:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "CentOS"

    - name: "MariaDB - Restarting/Enabling"
      service:
        name: mariadb
        state: restarted
        enabled: true

    - name: install mariadb package (Ubuntu)
      apt:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "Ubuntu"
  
```

Make sure to save the file and exit.

```

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]

TASK [MariaDB - Restarting/Enabling] *****
fatal: [192.168.56.106]: FAILED! => {"changed": false, "msg": "Could not find the requested service mariadb: host"}

PLAY RECAP *****
192.168.56.103      : ok=4   changed=0    unreachable=0    failed=0    skipped=2
rescued=0   ignored=0
192.168.56.104      : ok=4   changed=0    unreachable=0    failed=0    skipped=2
rescued=0   ignored=0
192.168.56.106      : ok=3   changed=0    unreachable=0    failed=1    skipped=2
rescued=0   ignored=0
192.168.56.107      : ok=2   changed=0    unreachable=0    failed=0    skipped=1
rescued=0   ignored=0
  
```

```
ok: [192.168.56.104]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.104]
ok: [192.168.56.103]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]

TASK [install mariadb package (Ubuntu)] *****
changed: [192.168.56.106]

TASK [MariaDB - Restarting/Enabling] *****
changed: [192.168.56.106]

PLAY RECAP *****
192.168.56.103      : ok=4   changed=0   unreachable=0   failed=0   skipped=2
rescued=0   ignored=0
192.168.56.104      : ok=4   changed=0   unreachable=0   failed=0   skipped=2
rescued=0   ignored=0
192.168.56.106      : ok=5   changed=2   unreachable=0   failed=0   skipped=2
rescued=0   ignored=0
192.168.56.107      : ok=2   changed=0   unreachable=0   failed=0   skipped=1
rescued=0   ignored=0

mdmlot@workstation:~/CPE212_loterte$
```

Run the site.yml file and describe the result

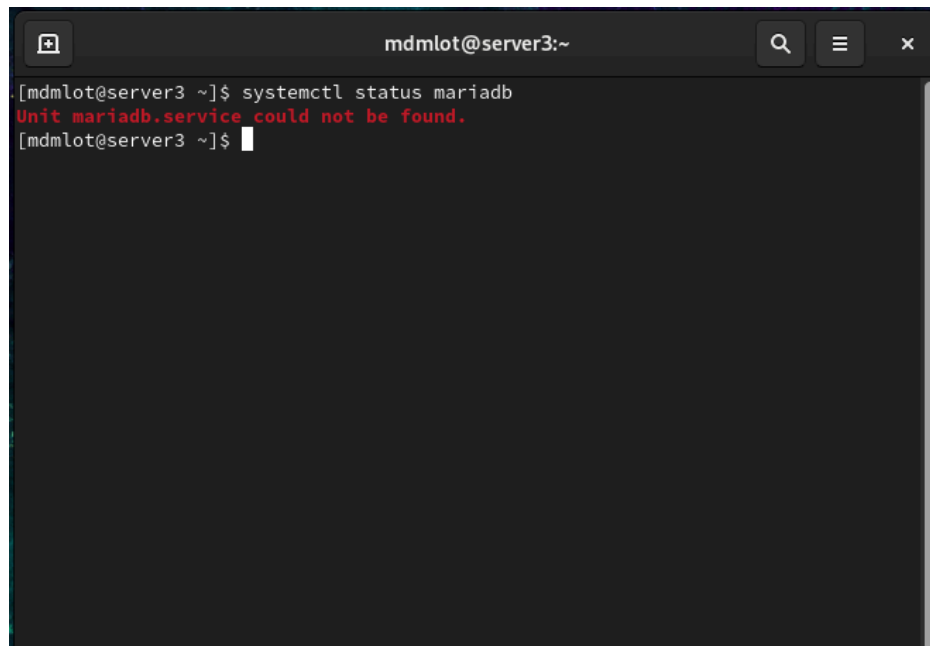
The site.yml file initially resulted into an error which was fixed by simply moving the restarting and enabling of mariadb after the Ubuntu mariaDB installation. Because the CentOS is set only to web servers and only 192.168.56.106 was set as a db server, only in 192.168.56.106 was mariaDB installed.

5. Go to the remote server (Ubuntu) terminal that belongs to the db_servers group and check the status for mariadb installation using the command: `systemctl status mariadb`. Do this on the CentOS server also.

```
mdmlot@server2: ~
inet 192.168.56.106 netmask 255.255.255.0 broadcast 192.168.56.255
inet6 fe80::58e:1d54:5423:f6cf prefixlen 64 scopeid 0x20<link>
ether 08:00:27:de:08:23 txqueuelen 1000 (Ethernet)
RX packets 2828 bytes 2489816 (2.4 MB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 974 bytes 172238 (172.2 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 410 bytes 50141 (50.1 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 410 bytes 50141 (50.1 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mdmlot@server2:~$ systemctl status mariadb
● mariadb.service - MariaDB 10.11.14 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset:
   Active: active (running) since Wed 2026-09-02 20:59:21 PST; 3min 59s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 31195 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /va
```

A screenshot of a terminal window titled 'mdmlot@server3:~'. The window has a dark background and a light-colored border. The terminal shows the command 'systemctl status mariadb' being executed. The output is 'Unit mariadb.service could not be found.' in red text. The prompt '[mdmlot@server3 ~]\$' is visible at the bottom of the terminal.

```
mdmlot@server3 ~]$ systemctl status mariadb
Unit mariadb.service could not be found.
[mdmlot@server3 ~]$
```

Describe the output.

Only in the Ubuntu server with the IP address of 192.168.56.106 was MariaDB installed. The only CentOS installation I have does not contain mariaDB because it was not set as part of db_servers in the hosts inventory.yaml.

6. Edit the site.yml again. This time we will append the code to configure installation on the file_servers group. We can add the following on our file.

```

GNU nano 7.2                                site.yml
tasks:
- name: install mariadb package (CentOS)
  yum:
    name: mariadb-server
    state: latest
    when: ansible_distribution == "CentOS"

- name: install mariadb package (Ubuntu)
  apt:
    name: mariadb-server
    state: latest
    when: ansible_distribution == "Ubuntu"

- name: "MariaDB - Restarting/Enabling"
  service:
    name: mariadb
    state: restarted
    enabled: true

- hosts: file_servers
  become: true
  tasks:
    - name: install samba package
      package:
        name: samba
        state: latest
  
```

[Wrote 66 lines]

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^_ Replace ^U Paste ^J Justify ^_ Go To Line

Make sure to save the file and exit.

```

TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.106]

TASK [MariaDB - Restarting/Enabling] *****
changed: [192.168.56.106]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.107]

TASK [install samba package] *****
changed: [192.168.56.107]

PLAY RECAP *****
192.168.56.103      : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
192.168.56.104      : ok=4    changed=0    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
192.168.56.106      : ok=5    changed=1    unreachable=0    failed=0    skipped=2
rescued=0    ignored=0
192.168.56.107      : ok=4    changed=1    unreachable=0    failed=0    skipped=1
rescued=0    ignored=0

mdmlot@workstation:~/CPE212_loterte$
  
```

Run the site.yml file and describe the result.

Like in previous procedures, samba was only installed to 192.168.56.107 because that was the only managed node set to file_servers. Because of package: in the site.yml, the play would've still worked with a CentOS

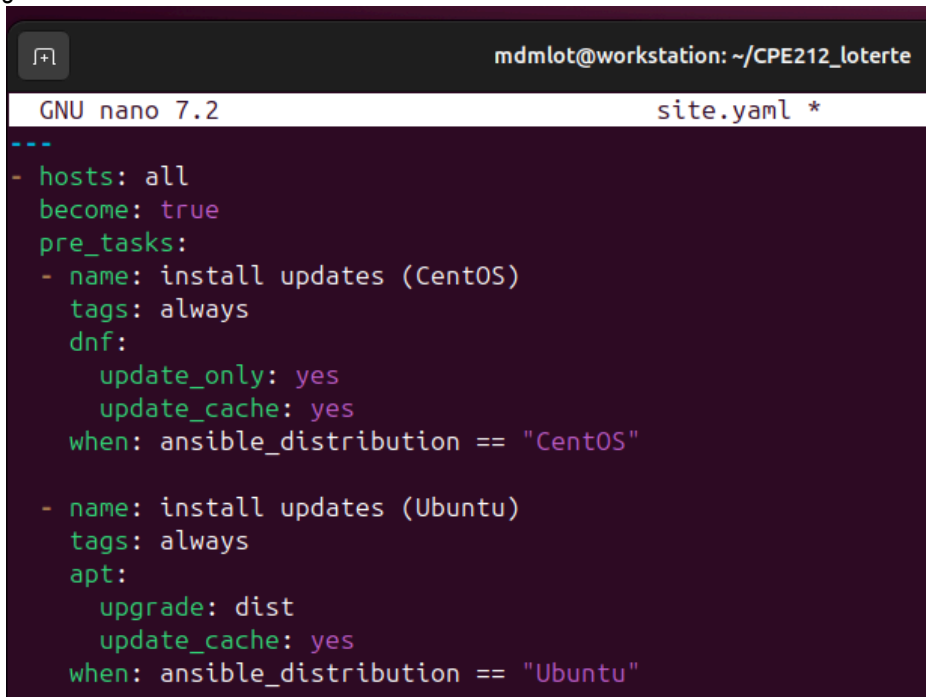
installation.

The testing of the file_servers is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we can target hosts or servers using grouping in ansible playbooks.

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the site.yml file. Add tags to the playbook. After the name, we can place the tags: name_of_tag. This is an arbitrary command, which means you can use any name for a tag.

A screenshot of a terminal window showing the nano text editor. The terminal title bar indicates the user is 'mdmlot' at 'workstation' in the directory '~/CPE212_loterte'. The editor shows the file 'site.yml' with two tasks. The first task is for CentOS, and the second is for Ubuntu. Both tasks include tags: 'always' and a distribution-specific tag ('CentOS' or 'Ubuntu').

```
mdmlot@workstation: ~/CPE212_loterte
GNU nano 7.2 site.yml *
---
- hosts: all
  become: true
  pre_tasks:
  - name: install updates (CentOS)
    tags: always
    dnf:
      update_only: yes
      update_cache: yes
    when: ansible_distribution == "CentOS"

  - name: install updates (Ubuntu)
    tags: always
    apt:
      upgrade: dist
      update_cache: yes
    when: ansible_distribution == "Ubuntu"
```

```
mdmlot@workstation: ~/CPE212_loterte
GNU nano 7.2 site.yaml *
```

```
- hosts: web_servers
  become: true
  tasks:

  - name: install apache and php for ubuntu servers
    tags: apache,apache2,ubuntu
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    tags: apache,httpd,centos
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

```
mdmlot@workstation: ~/CPE212_loterte
GNU nano 7.2 site.yaml *
```

```
- name: install mariadb package (CentOS)
  tags: centos,db,mariadb
  yum:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "CentOS"

- name: install mariadb package (Ubuntu)
  tags: ubuntu,db,mariadb
  apt:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "Ubuntu"

- name: "MariaDB - Restarting/Enabling"
  service:
    name: mariadb
    state: restarted
    enabled: true

- hosts: file_servers
  become: true
  tasks:
  - name: install samba package
    tags: samba
    package:
```

Make sure to save the file and exit.

```

mdmlot@workstation: ~/CPE212_loterte
TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.106]

TASK [MariaDB - Restarting/Enabling] *****
changed: [192.168.56.106]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.107]

TASK [install samba package] *****
ok: [192.168.56.107]

PLAY RECAP *****
192.168.56.103      : ok=4    changed=0    unreachable=0    failed=0    skipped=2
  rescued=0    ignored=0
192.168.56.104      : ok=4    changed=0    unreachable=0    failed=0    skipped=2
  rescued=0    ignored=0
192.168.56.106      : ok=5    changed=1    unreachable=0    failed=0    skipped=2
  rescued=0    ignored=0
192.168.56.107      : ok=4    changed=0    unreachable=0    failed=0    skipped=1
  rescued=0    ignored=0

mdmlot@workstation:~/CPE212_loterte$

```

Run the site.yml file and describe the result.

The process was similar as before because we haven't utilized tags yet to filter and specify which task groups are to be ran.

2. On the local machine, try to issue the following commands and describe each result:

2.1 ansible-playbook --list-tags site.yml

```

mdmlot@workstation:~/CPE212_loterte$ ansible-playbook --list-tags site.yml

playbook: site.yml

play #1 (all): all    TAGS: []
  TASK TAGS: [always]

play #2 (web_servers): web_servers    TAGS: []
  TASK TAGS: [apache, apache2, centos, httpd, ubuntu]

play #3 (db_servers): db_servers    TAGS: []
  TASK TAGS: [centos, db, mariadb, ubuntu]

play #4 (file_servers): file_servers    TAGS: []
  TASK TAGS: [samba]

```

Just as the name suggests, all the tags are listed out for display in a per-play basis.

2.2 ansible-playbook --tags centos --ask-become-pass site.yml

```
mdmlot@workstation: ~/CPE212_loterte
ok: [192.168.56.103]

TASK [install updates (CentOS)] *****
skipping: [192.168.56.104]
skipping: [192.168.56.106]
skipping: [192.168.56.107]
ok: [192.168.56.103]

TASK [install updates (Ubuntu)] *****
skipping: [192.168.56.103]
ok: [192.168.56.107]
ok: [192.168.56.104]
ok: [192.168.56.106]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.104]
ok: [192.168.56.103]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]
```

Pre-tasks still continued to run but tasks that did not contain the tag "centos" were skipped completely

2.3 ansible-playbook --tags db --ask-become-pass site.yml

```
TASK [install updates (Ubuntu)] *****
skipping: [192.168.56.103]
ok: [192.168.56.104]
ok: [192.168.56.107]
ok: [192.168.56.106]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.106]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.107]

PLAY RECAP *****
192.168.56.103 : ok=3 changed=0 unreachable=0 failed=0 skipped=1
```

After the pre-tasks, only tasks set for host: db_servers was played as they were the only ones with the db tag.

2.4 ansible-playbook --tags apache --ask-become-pass site.yml

```

skipping: [192.168.56.103]
ok: [192.168.56.104]
ok: [192.168.56.107]
ok: [192.168.56.106]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [install apache and php for ubuntu servers] *****
skipping: [192.168.56.103]
ok: [192.168.56.104]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.104]
ok: [192.168.56.103]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.107]

```

This time, only tasks meant for web_servers (besides pre-tasks) ran as they are the only ones with the tags apache.

2.5 ansible-playbook --tags "apache,db" --ask-become-pass site.yml

```

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.104]
ok: [192.168.56.103]

TASK [install apache and php for ubuntu servers] *****
skipping: [192.168.56.103]
ok: [192.168.56.104]

TASK [install apache and php for CentOS servers] *****
skipping: [192.168.56.104]
ok: [192.168.56.103]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [192.168.56.106]

TASK [install mariadb package (CentOS)] *****
skipping: [192.168.56.106]

TASK [install mariadb package (Ubuntu)] *****
ok: [192.168.56.106]

PLAY [file_servers] *****

TASK [Gathering Facts] *****

```

For this iteration, the tasks for web_servers and db_servers were ran thanks to the listing of tags using quotations and commas. This also means that tasks for file_servers did not run because they were not included within the quoted tags.

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

```
GNU nano 6.2 site.yml *
- libapache2-mod-php
  state: latest
  update_cache: yes
  when: ansible_distribution == "Ubuntu"

- name: install apache and php for CentOS servers
  tags: apache,httpd,centos
  dnf:
    name:
      - httpd
      - php
    state: latest
  when: ansible_distribution == "CentOS"

- name: start httpd (CentOS)
  tags: apache,centos,httpd
  service:
    name: httpd
    state: started
  when: ansible_distribution == "CentOS"

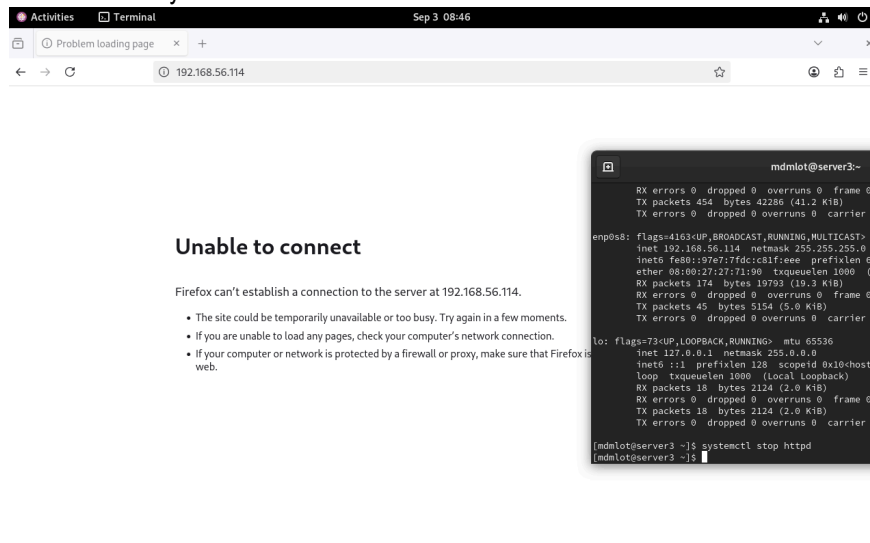
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Lin
```

Make sure to save the file and exit.

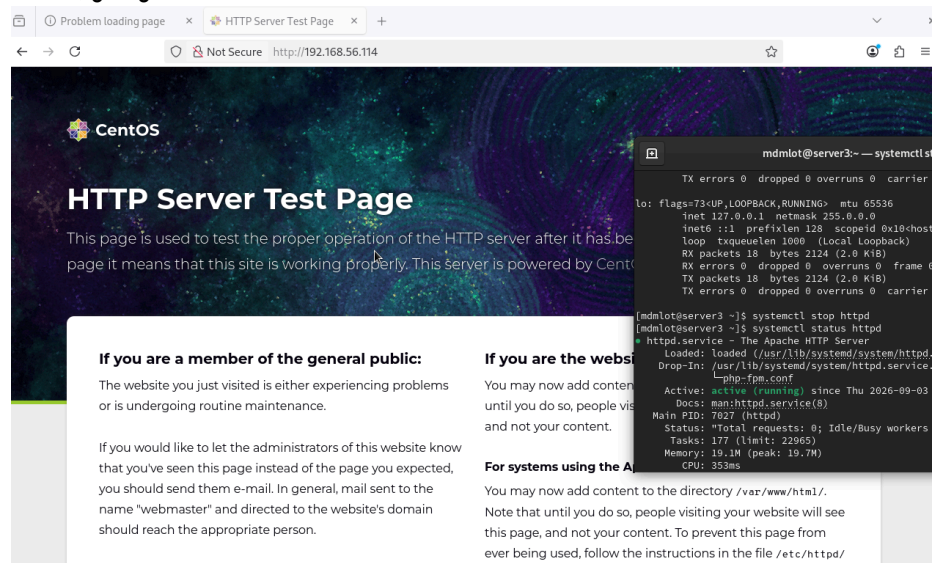
You would also notice from our previous activity that we already created a module that runs a service

This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command `sudo systemctl stop httpd`. When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.



3. Go to the local machine and this time, run the site.yml file. Then after running the file, go again to the CentOS server and enter its IP address on the browser.



Describe the result.

After running the playbook with the `--tags httpd` option to run it faster, refreshing the initial web page still resulted in an error even when the service itself is open. Opening a new web page and putting in the same IP address will result in the correct test page appearing.

To automatically enable the service every time we run the playbook, use the command enabled: true similar to Figure 7.1.2 and save the playbook.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. Creating a Role for Web Servers

```
mdmlot@workstation:~/CPE212_loterte$ ansible-galaxy init web_servers --offline
- Role web_servers was created successfully
mdmlot@workstation:~/CPE212_loterte$ ls
install_apache.yml  inventory.yml  README.md  site.yml  web_servers
```

2. Creating a Role for Database Servers

```
mdmlot@workstation:~/CPE212_loterte$ ansible-galaxy init db_servers --offline
- Role db_servers was created successfully
mdmlot@workstation:~/CPE212_loterte$ ls
db_servers  install_apache.yml  inventory.yml  README.md  site.yml  web_servers
mdmlot@workstation:~/CPE212_loterte$
```

3. Creating a Role for File Servers

```
mdmlot@workstation:~/CPE212_loterte$ nano site.yml
mdmlot@workstation:~/CPE212_loterte$ ansible-galaxy init file_servers --offline
- Role file_servers was created successfully
mdmlot@workstation:~/CPE212_loterte$ ls
db_servers  file_servers  install_apache.yml  inventory.yml  README.md  site.yml  web_servers
mdmlot@workstation:~/CPE212_loterte$
```

4. Combining Multiple Roles in a Playbook

To orchestrate multiple roles in a single playbook for different server groups. Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for database.

```
mdmlot@workstation: ~/CPE212_loterte
GNU nano 7.2          hoa6_supp.yaml
tags: always
dnf:
  update_only: yes
  update_cache: yes
when: ansible_distribution == "CentOS"

- name: install updates (Ubuntu)
  tags: always
  apt:
    upgrade: dist
    update_cache: yes
  when: ansible_distribution == "Ubuntu"

- hosts: web_servers
  become: true
  roles:
    - web_servers
  tags: web

- hosts: db_servers
  become: true
  roles:
    - db_servers
  tags: db

- hosts: file_servers
  become: true
  roles:
    - file_servers
  tags: file
```

Reflections:

Answer the following:

1. What is the importance of putting our remote servers into groups?

Grouping remote servers using metagroups and roles allows us to do targeted deployment, skipping over unrequired tasks within a playbook. It allows for scalability by reducing the need for editing individual playbooks.

2. What is the importance of tags in playbooks?

Tags also play into Ansible's ability of targeted deployment through selective execution of tasks containing said tags, saving us large amounts of time from running redundant or unneeded tasks.

3. Why do think some services need to be managed automatically in playbooks?

Automating the management of services via playbooks reduces the need of tedious double-checking manually over every server. It also ensures high availability of these required services alongside the constant runtime of these servers to avoid problems appearing on the clients' end.

Conclusions:

I conclude that targeting specific nodes using Ansible playbooks by grouping and tagging them allows for efficient, automated management across multiple servers. Using specific groups and tags ensure that the system is scalable and playbook runtime across multiple servers do their required functions for the least amount of time.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.